CODES:

BankModules.v:

```
/*
 * Yigit Suoglu
 * Bank Queue Management System
 * Designed to work on Digilent Basys 3 but it should work on any FPGA
 * Size of password switches and leds output can be changed without effecting functionality
 * Licence: CERN-OHL-W
 */
`timescale 1ns / 1ps

module bankSystem(ssd0, ssd1, ssd2, ssd3, passSW, Teller1btn, Teller0btn,
  bankLed, Teller1led, Teller0led, rst, clk, customerBTN, counterSW, bankSW, chPASS, Wait);


  input clk, rst; //clock and reset
  output [4:0] ssd1, ssd2, ssd3, ssd0; //what each ssd should be in abcdefg format
  input [11:0] passSW; //password switches
  input Teller1btn, Teller0btn; //teller buttons
  input customerBTN; //button for customer to take number
  input counterSW; //open-close counter
  input bankSW; //open-close bank
  input chPASS; //change password
  output reg bankLed, Teller1led, Teller0led; //bank & counter open leds
  reg [7:0] callingCus, totalCus, teller1call, teller0call;
  //output [12:0] leds; //other leds to show stage
  reg [11:0] passT1, passT0; //holds passwords for tellers default 12'd0;
  reg trigQueNumTake; //trigger to show taken Queue number
  output reg Wait; //waiting something, works with operation signal
  reg waitswho; //Shows system waitng who, work with wait signal

  wire blinkOff, queueActive;
  wire [7:0] waitingCus; //remaning customer
  wire readyForClose; //no customer left and both counters are closed
  wire [2:0] operation; //combined operation switches
  wire notRdyCls;

  assign notRdyCls = ~readyForClose & bankSW;
  assign operation = {bankSW, counterSW, chPASS}; //order: bank, counter, change password
  assign readyForClose = (~(Teller1led | Teller0led) & (callingCus == totalCus)) | ~bankLed;
  assign waitingCus = totalCus - callingCus;

  always@(posedge clk or posedge rst) //Calling customer
    begin
      if(rst)
        begin
          teller1call <= 8'b0;
          teller0call <= 8'b0;
          callingCus <= 8'b0;
        end
      else if((operation == 3'b0) & (waitingCus != 8'b0))
        begin
        /* if(Teller0btn & Teller1btn & Teller1led & Teller0led) //both tellers call at same time cycle, not necessary
            begin
              teller1call <= callingCus + 8'b10;
              teller0call <= callingCus + 8'b1;
              callingCus <= callingCus + 8'b10;
            end
          else */if(Teller0btn & Teller0led)
            begin
              teller0call <= callingCus + 8'b1;
              callingCus <= callingCus + 8'b1;
            end
          else if(Teller1btn & Teller1led)
            begin
              teller1call <= callingCus + 8'b1;
              callingCus <= callingCus + 8'b1;
            end
        end
    end

  always@(posedge clk or posedge rst) //operations
    begin
      if(rst)
        begin
          bankLed <= 0;
          Teller0led <= 0;
          Teller1led <= 0;
          Wait <= 0;
          passT0 <= 12'd0;
          passT1 <= 12'd0;
        end
      else
        case (operation)
          3'b100: //bank open-close
            begin //#1
              if(readyForClose)
                begin //#2
                  if (Wait) //if one of the tellers entered pasword
                      case(waitswho)
                        0: if((passSW == passT0) & Teller0btn)
                            begin //#3
                              bankLed <= ~bankLed;
                              Wait <= 0;
                            end //#3
                          1: if((passSW == passT1) & Teller1btn)
                            begin //#3
                              bankLed <= ~bankLed;
```

```
                                Wait <= 0;
                              end //#3
                        endcase
                    else
                      begin //#3
                        if((passSW == passT0) & Teller0btn)
                          begin //#4
                            Wait <= 1;
                            waitswho <= 1;
                          end //#4
                        else if((passSW == passT1) & Teller1btn)
                          begin //#4
                            Wait <= 1;
                            waitswho <= 0;
                          end //#4
                      end //#3
                    end //#2
              end //#1
            3'b010: //counter open-close
              begin
                if((passSW == passT0) & Teller0btn)
                    Teller0led <= ~Teller0led;
                else if((passSW == passT1) & Teller1btn)
                    Teller1led <= ~Teller1led;
              end
            3'b001: //change password
              begin
                if (Wait) //if one of the tellers entered pasword
                    case(waitswho)
                      0: if(Teller0btn)
                          passT0 <= passSW;
                      1: if(Teller1btn)
                          passT1 <= passSW;
                    endcase
                else
                  begin //#3
                    if((passSW == passT0) & Teller0btn)
                      begin //#4
                        Wait <= 1;
                        waitswho <= 0;
                      end //#4
                    else if((passSW == passT1) & Teller1btn)
                      begin //#4
                        Wait <= 1;
                        waitswho <= 1;
                      end //#4
                  end //#3
              end
```

```
            default: Wait <= 0;
          endcase
      end

    always@(posedge clk or posedge rst) //Queue number taking
      begin
        if (rst)
          begin
            trigQueNumTake <= 0;
            totalCus <= 8'd0;
          end
        else if(bankLed)
          begin
            if(customerBTN)
              begin
                trigQueNumTake <= 1;
                totalCus <= totalCus + 8'b1;
              end
            else
              begin
                trigQueNumTake <= 0;
              end
          end
        else
          begin
            totalCus <= 8'd0;
            trigQueNumTake <= 0;
          end
      end

    displaySelection ssdSelector(.dig0(ssd0), .dig1(ssd1), .dig2(ssd2), .dig3(ssd3),
        .bankClosed(bankLed), .queueNum(queueActive), .totalCus(totalCus),
        .teller1call(teller1call), .teller0call(teller0call),
        .notRdyCls(notRdyCls), .waitingCus(waitingCus), .blinkOff(blinkOff));

    queueNumTakeProces queuemng(.trigQueNumTake(trigQueNumTake), .clk(clk), .rst(rst),
                                .queueActive(queueActive), .blinkOff(blinkOff));
  endmodule

module displaySelection(dig0, dig1, dig2, dig3, bankClosed, queueNum, totalCus,
    teller1call, teller0call, notRdyCls, waitingCus, blinkOff/*, pswrdCH*/);
    parameter a = 5'd10, b = 5'd11, c = 5'd12, d = 5'd13, h = 5'd19;
    parameter e = 5'd14, f = 5'd15, o = 5'd16, L = 5'd17, p = 5'd18, r = 5'd20;

    output reg [4:0] dig0, dig1, dig2, dig3;
    input bankClosed, notRdyCls;
    input blinkOff, queueNum/*, pswrdCH*/;
```

```
    input [7:0] totalCus, teller1call, teller0call, waitingCus;

    always@(*)
      if(!bankClosed) //bank closed
        begin
          dig3 = c;
          dig2 = L;
          dig1 = 5'd5; //s
          dig0 = d;
        end
      else if(blinkOff) //ssd's are not ilmunated (used for blinking)
        begin
          dig3 = 5'b11111;
          dig2 = 5'b11111;
          dig1 = 5'b11111;
          dig0 = 5'b11111;
        end
      else if(queueNum) //show taken number
        begin
          dig3 = 5'b11111;
          dig2 = 5'b11111;
          dig1 = {1'b0 ,totalCus[7:4]};
          dig0 = {1'b0 ,totalCus[3:0]};
        end
      else if(notRdyCls) //show taken number
        begin
          dig3 = r;
          dig2 = 5'd12; //c
          dig1 = {1'b0 ,waitingCus[7:4]};
          dig0 = {1'b0 ,waitingCus[3:0]};
        end
    /*  else if(pswrdCH) //ch pasword mode
        begin
          dig3 = 5'd12; //c
          dig2 = h;
          dig1 = p;//
          dig0 = 5'd5; //s
        end */
      else
        begin //last called number
          dig3 = {1'b0 ,teller1call[7:4]};
          dig2 = {1'b0 ,teller1call[3:0]};
          dig1 = {1'b0 ,teller0call[7:4]};
          dig0 = {1'b0 ,teller0call[3:0]};
        end
  endmodule

  module queueNumTakeProces(trigQueNumTake, clk, rst, queueActive, blinkOff);
    input trigQueNumTake, clk, rst;
    output reg queueActive;
    output blinkOff;
    reg [1:0] state;
    wire oneSecSig, quarSecSig, quaTrig, quaTrigRep;
    reg quarSecSig_s;

    quarSecWait quaCount(.start(quaTrig), .rst(rst), .clk(clk), .lastCycle(quarSecSig));
    oneSecWait oneCount(.start(trigQueNumTake), .rst(rst), .clk(clk), .lastCycle(oneSecSig));

    assign quaTrig = trigQueNumTake | quaTrigRep;
    assign blinkOff = state[0];

    always@(posedge clk or posedge rst) //state transaction
      begin
        if (rst)
          begin
            state <= 2'b0;
          end
        else
          case (state)
            2'b11: state <= 2'b0;
            default: state <= state + {1'b0, quarSecSig};
          endcase
      end

    always@(posedge clk or posedge rst)//queueActive signal
      if (rst)
          queueActive <= 0;
      else if (oneSecSig)
          queueActive <= 0;
      else if (trigQueNumTake)
          queueActive <= 1;

      assign quaTrigRep = quarSecSig_s & (state != 2'd3);

      always@(posedge clk)
          begin
              quarSecSig_s <= quarSecSig;
          end


  endmodule
```

Counters.v:

```
 //Yigit Suoglu
//This file contains counters to count 1 sec, 0.5 sec and 0.25 sec.
//Note: parameters should be updated according to device clock
//Parameters are for clock frequency of 100MHz (period of 10 ns)
// Licence: CERN-OHL-W
`timescale 1ns / 1ps

module oneSecWait(start, rst, clk, lastCycle);
  parameter oneSec = 27'd100000000; //frequency of device clock, use 4'b1000 for sim
  parameter counterSize = 27; //size of reg, use 4 for sim

  input start, rst, clk;
  output lastCycle;

  reg [(counterSize - 1):0] count;

  assign lastCycle = (count == oneSec);

  always@(posedge clk or posedge rst)
    begin
      if(rst)
        count <= 0;
      else if(count == oneSec)
        count <= 0;
      else if(start)
        count <= 1;
      else if(count != 0)
        count <= count + 1;
      else
        count <= count;
    end

endmodule //1 sec

module halfSecWait(start, rst, clk, lastCycle);
  parameter oneSec = (27'd100000000 >> 1); //frequency of device clock, use 3'b100 for sim
  parameter counterSize = (27 - 1); //size of reg, use 3 for sim

  input start, rst, clk;
  output lastCycle;

  reg [(counterSize - 1):0] count;

  assign lastCycle = (count == oneSec);

  always@(posedge clk or posedge rst)
    begin
      if(rst)
        count <= 0;
      else if(count == oneSec)
        count <= 0;
      else if(start)
        count <= 1;
      else if(count != 0)
        count <= count + 1;
      else
        count <= count;
    end

endmodule //0.5 sec

module quarSecWait(start, rst, clk, lastCycle);
  parameter oneSec = (27'd100000000 >> 2); //frequency of device clock, use 2'b10 for sim
  parameter counterSize = (27 - 2); //size of reg, use 2 for sim

  input start, rst, clk;
  output lastCycle;

  reg [(counterSize - 1):0] count;


  always@(posedge clk or posedge rst)
    begin
      if(rst)
        count <= 0;
      else if(count == oneSec)
        count <= 0;
      else if(start)
        count <= 1;
      else if(count != 0)
        count <= count + 1;
      else
        count <= count;
    end

  assign lastCycle = (count == oneSec);


endmodule //0.25 sec
```

SevenSegmentDisplayDecoders.v:

```
/*
 *  Yigit Suoglu
 *    Licence: CERN-OHL-W
 *    Out signals in form of abcdefg corresponding:
 *      a
 *    f   b         Note: abcdefg signals should driven low to ilmunate
 *      g                 corresponding segment.
 *    e   c
 *      d
 */
`timescale 1ns / 1ps

module int4bitToHexSSD(in, out);
  parameter zero = 7'b0000001, one = 7'b1001111, two = 7'b0010010;
  parameter thr = 7'b0000110, four = 7'b1001100, five = 7'b0100100;
  parameter six = 7'b0100000, svn = 7'b0001111, eght = 7'b0000000;
  parameter nine = 7'b0000100, A = 7'b0001000, B = 7'b1100000;
   parameter C = 7'b0110001, D = 7'b1000010, E = 7'b0110000, F = 7'b0111000;


  input [3:0] in;
  output reg [6:0] out;

  always@(*)
        case (in)
          4'b0: out = zero;
          4'b1: out = one;
          4'b10: out = two;
          4'b11: out = thr;
          4'b100: out = four;
          4'b101: out = five;
          4'b110: out = six;
          4'b111: out = svn;
          4'b1000: out = eght;
          4'b1001: out = nine;
          4'b1010: out = A;
          4'b1011: out = B;
          4'b1100: out = C;
          4'b1101: out = D;
          4'b1110: out = E;
          4'b1111: out = F;

        endcase
endmodule

module int5bitToHexSSD_bankVer(in, out);
  parameter zero = 7'b0000001, one = 7'b1001111, two = 7'b0010010; //in = 0, 1, 2
  parameter thr = 7'b0000110, four = 7'b1001100, five = 7'b0100100; //in = 3, 4, 5 ~ 5 = s in ssd
  parameter six = 7'b0100000, svn = 7'b0001111, eght = 7'b0000000; //in = 6, 7, 8
  parameter nine = 7'b0000100, A = 7'b0001000, B = 7'b1100000; //in = 9, 10, 11
  parameter C = 7'b0110001, D = 7'b1000010, E = 7'b0110000, F = 7'b0111000; //in = 12, 13, 14, 15
  parameter  o = 7'b1100010, L = 7'b1110001, empty = 7'b1111111; //in = 16, 17, others
  parameter  p = 7'b0011000, h = 7'b1101000, r = 7'b1111010; //in = 18, 19, 20



  input [4:0] in;
  output reg [6:0] out;

  always@(*)
        case (in)
          5'b0: out = zero;
          5'b1: out = one;
          5'b10: out = two;
          5'b11: out = thr;
          5'b100: out = four;
          5'b101: out = five;
          5'b110: out = six;
          5'b111: out = svn;
          5'b1000: out = eght;
          5'b1001: out = nine;
          5'b1010: out = A;
          5'b1011: out = B;
          5'b1100: out = C;
          5'b1101: out = D;
          5'b1110: out = E;
          5'b1111: out = F;
          5'b10000: out = o;
          5'b10001: out = L;
          5'b10010: out = p;
          5'b10011: out = h;
          5'b10100: out = r;
          default: out = empty;
        endcase
endmodule
```

board.v:

```
`timescale 1ns / 1ps

module board(
    input clk,
    input rst,
    input [11:0] passSW,
    input T1btn,
    input T0btn,
    input Cbtn,
    output bnkLED,
    output t1LED,
    output t0LED,
    input counterSW,
    input bankSW,
    input chPASS,
    output a,
    output b,
    output c,
    output d,
    output e,
    output f,
    output g,
    output an0,
    output an1,
    output an2,
    output an3,
    output Wait
    );

    wire [4:0] ssd0_pre, ssd1_pre, ssd2_pre, ssd3_pre;
    wire [6:0] ssd0, ssd1, ssd3, ssd2;
    wire  T1btn_c, T0btn_c,  Cbtn_c;

    bankSystem core(.ssd0(ssd0_pre), .ssd1(ssd1_pre), .ssd2(ssd2_pre), .ssd3(ssd3_pre), .passSW(passSW),
    .Teller1btn(T1btn_c), .Teller0btn(T0btn_c), .bankLed(bnkLED), .Teller1led(t1LED), .Teller0led(t0LED),
    .rst(rst), .clk(clk), .customerBTN(Cbtn_c),  .counterSW(counterSW), .bankSW(bankSW), .chPASS(chPASS), .Wait(Wait));

    int5bitToHexSSD_bankVer ssd0M(.in(ssd0_pre), .out(ssd0));
    int5bitToHexSSD_bankVer ssd1M(.in(ssd1_pre), .out(ssd1));
    int5bitToHexSSD_bankVer ssd2M(.in(ssd2_pre), .out(ssd2));
    int5bitToHexSSD_bankVer ssd3M(.in(ssd3_pre), .out(ssd3));

    ssd ssdControl(.clk(clk), .rst(rst), .a0(ssd0[0]), .a1(ssd1[0]), .a2(ssd2[0]), .a3(ssd3[0]),
    .b0(ssd0[1]), .b1(ssd1[1]), .b2(ssd2[1]), .b3(ssd3[1]), .c0(ssd0[2]), .c1(ssd1[2]), .c2(ssd2[2]), .c3(ssd3[2]),
    .d0(ssd0[3]), .d1(ssd1[3]), .d2(ssd2[3]), .d3(ssd3[3]), .e0(ssd0[4]), .e1(ssd1[4]), .e2(ssd2[4]), .e3(ssd3[4]),
    .f0(ssd0[5]), .f1(ssd1[5]), .f2(ssd2[5]), .f3(ssd3[5]), .g0(ssd0[6]), .g1(ssd1[6]), .g2(ssd2[6]), .g3(ssd3[6]),
    .a(a), .b(b), .c(c), .d(d), .e(e), .f(f), .g(g), .an0(an0), .an1(an1), .an2(an2), .an3(an3));

    debouncer t0BUT(.clk(clk), .rst(rst), .noisy_in(T0btn), .clean_out(T0btn_c));
    debouncer t1BUT(.clk(clk), .rst(rst), .noisy_in(T1btn), .clean_out(T1btn_c));
    debouncer cBUT(.clk(clk), .rst(rst), .noisy_in(Cbtn), .clean_out(Cbtn_c));

endmodule
```

debouncer.v:

```
`timescale 1ns / 1ps

module debouncer(
    input       clk,
    input       rst,
    input       noisy_in, // port from the push button
    output reg  clean_out // port into the circuit
     );


 reg noisy_in_reg;
 reg clean_out_tmp1; // will be used to detect rising edge
 reg clean_out_tmp2; // will be used to detect rising edge

 always@(posedge clk or posedge rst)
begin
    if (rst)
        begin
            noisy_in_reg <= 0;
            clean_out_tmp1 <= 0;
            clean_out_tmp2 <= 0;
            clean_out <= 0;
        end
    else
       begin
       noisy_in_reg <= noisy_in; // store the input
       clean_out_tmp1 <= noisy_in_reg;

       // rising edge detect
       clean_out_tmp2 <= clean_out_tmp1;
       clean_out <= ~clean_out_tmp2 & clean_out_tmp1; // it produce a single pulse during a risingedge
    end

 end
 endmodule
```

ssd.v:

```
`timescale 1ns / 1ps

module ssd( clk, rst, a0,a1,a2,a3,b0,b1,b2,b3,c0,c1,c2,c3,d0,d1,d2,d3,e0,e1,e2,e3,
f0,f1,f2,f3,g0,g1,g2,g3,a,b,c,d,e,f,g,an0,an1,an2,an3
    );

 input clk, rst;// set clock and reset as input(1 bit)
 input a0,a1,a2,a3,b0,b1,b2,b3,c0,c1,c2,c3,d0,d1,d2,d3,e0,e1,e2,e3,f0,f1,f2,f3,g0,g1,g2,g3;//set our display data as input(1 bit)
 output reg a,b,c,d,e,f,g,an0,an1,an2,an3;//set outputs also as registers (1 bit)

 reg [2:0] state;//holds state number (3 bit)
 reg [14:0] counter;//counter to slow the input clock(15 bit)

 //in this always block the speed of the clock reduced by 25000 times so that display works properly
 //(for a clock frequency is diffent than 100MHz use a counter value that gives reduced clock frequency if ~4kHz.
 //Line 12 and 23 should be changed accordingly)
always @ (posedge clk) begin //state counter
   if(rst) begin  //synchronus reset
       state <= 0;    //if reset set state and counter to zero
       counter <= 0;
   end else begin //else the counter untill 25000 (limit)

       if(counter == 15'h61A8) //if equal to 25000 (limit)
       begin
           if (state == 3'b100) //if it is in the last state return to state 1
           state <= 1;
           else                 //else go one state up
           state <= state + 1;

       counter <= 0;     //0 the counter because after it reaches 25000
       end
       else
       counter <= counter + 1; //if not 25000 add 1
   end
end


 always@(posedge clk)
begin
    if(rst)// if reset initilize the outputs
    begin
    an0 <= 1;
    an1 <= 1;
    an2 <= 1;
    an3 <= 1;
    a <= 1;
    b <= 1;
    c <= 1;
```

```
48      c <= 1;
49      d <= 1;
50      e <= 1;
51      f <= 1;
52      g <= 1;
53    end
54    else if(state == 3'b001) //state 1 gives the led outputs to the AN0
55    begin
56      an0 <= 0;
57      an1 <= 1;
58      an2 <= 1;
59      an3 <= 1;
60      a <= a0;
61      b <= b0;
62      c <= c0;
63      d <= d0;
64      e <= e0;
65      f <= f0;
66      g <= g0;
67    end
68    else if(state == 3'b010) //state 2 gives the led outputs to the AN1
69    begin
70      an0 <= 1;
71      an1 <= 0;
72      an2 <= 1;
73      an3 <= 1;
74      a <= a1;
75      b <= b1;
76      c <= c1;
77      d <= d1;
78      e <= e1;
79      f <= f1;
80      g <= g1;
81    end
82    else if(state == 3'b011) //state 3 gives the led outputs to the AN2
83    begin
84      an0 <= 1;
85      an1 <= 1;
86      an2 <= 0;
87      an3 <= 1;
88      a <= a2;
89      b <= b2;
90      c <= c2;
91      d <= d2;
92      e <= e2;
93      f <= f2;
94      g <= g2;
95    end
96    else if(state == 3'b100)   //state 4 gives the led outputs to the AN3
97    begin
98      an0 <= 1;
99      an1 <= 1;
100     an2 <= 1;
101     an3 <= 0;
102     a <= a3;
103     b <= b3;
104     c <= c3;
105     d <= d3;
106     e <= e3;
107     f <= f3;
108     g <= g3;
109   end
110   else //For other states default inputs and outputs
111   begin
112     an0 <= 1;
113     an1 <= 1;
114     an2 <= 1;
115     an3 <= 1;
116     a <= 1;
117     b <= 1;
118     c <= 1;
119     d <= 1;
120     e <= 1;
121     f <= 1;
122     g <= 1;
123   end
124 end
125 endmodule
```

cons.xdc:

```
## This file is a general .xdc for the Basys3 rev B board
## To use it in a project:
## - uncomment the lines corresponding to used pins
## - rename the used ports (in each line, after get_ports) according to the top level signal names in the project

## Clock signal
set_property PACKAGE_PIN W5 [get_ports clk]
   set_property IOSTANDARD LVCMOS33 [get_ports clk]
   create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]

## Switches
set_property PACKAGE_PIN V17 [get_ports {passSW[0]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {passSW[0]}]
set_property PACKAGE_PIN V16 [get_ports {passSW[1]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {passSW[1]}]
set_property PACKAGE_PIN W16 [get_ports {passSW[2]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {passSW[2]}]
set_property PACKAGE_PIN W17 [get_ports {passSW[3]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {passSW[3]}]
set_property PACKAGE_PIN W15 [get_ports {passSW[4]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {passSW[4]}]
set_property PACKAGE_PIN V15 [get_ports {passSW[5]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {passSW[5]}]
set_property PACKAGE_PIN W14 [get_ports {passSW[6]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {passSW[6]}]
set_property PACKAGE_PIN W13 [get_ports {passSW[7]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {passSW[7]}]
set_property PACKAGE_PIN V2 [get_ports {passSW[8]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {passSW[8]}]
set_property PACKAGE_PIN T3 [get_ports {passSW[9]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {passSW[9]}]
set_property PACKAGE_PIN T2 [get_ports {passSW[10]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {passSW[10]}]
set_property PACKAGE_PIN R3 [get_ports {passSW[11]}]
   set_property IOSTANDARD LVCMOS33 [get_ports {passSW[11]}]

set_property PACKAGE_PIN U1 [get_ports {chPASS}]
   set_property IOSTANDARD LVCMOS33 [get_ports {chPASS}]
set_property PACKAGE_PIN T1 [get_ports {counterSW}]
   set_property IOSTANDARD LVCMOS33 [get_ports {counterSW}]
set_property PACKAGE_PIN R2 [get_ports {bankSW}]
   set_property IOSTANDARD LVCMOS33 [get_ports {bankSW}]


## LEDs
set_property PACKAGE_PIN U16 [get_ports {t0LED}]
   set_property IOSTANDARD LVCMOS33 [get_ports {t0LED}]
set_property PACKAGE_PIN E19 [get_ports {t1LED}]
   set_property IOSTANDARD LVCMOS33 [get_ports {t1LED}]

set_property PACKAGE_PIN V14 [get_ports {Wait}]
   set_property IOSTANDARD LVCMOS33 [get_ports {Wait}]

set_property PACKAGE_PIN L1 [get_ports {bnkLED}]
   set_property IOSTANDARD LVCMOS33 [get_ports {bnkLED}]


##7 segment display
# NOTE: abcdefg signals are connected in reverse order
set_property PACKAGE_PIN W7 [get_ports {g}] #normally a
   set_property IOSTANDARD LVCMOS33 [get_ports {g}] #normally a
set_property PACKAGE_PIN W6 [get_ports {f}] #normally  b
   set_property IOSTANDARD LVCMOS33 [get_ports {f}] #normally b
set_property PACKAGE_PIN U8 [get_ports {e}] #normally c
   set_property IOSTANDARD LVCMOS33 [get_ports {e}] #normally c
set_property PACKAGE_PIN V8 [get_ports {d}]
   set_property IOSTANDARD LVCMOS33 [get_ports {d}]
set_property PACKAGE_PIN U5 [get_ports {c}] #normally e
   set_property IOSTANDARD LVCMOS33 [get_ports {c}] #normally e
set_property PACKAGE_PIN V5 [get_ports {b}] #normally f
   set_property IOSTANDARD LVCMOS33 [get_ports {b}] #normally f
set_property PACKAGE_PIN U7 [get_ports {a}] #normally g
   set_property IOSTANDARD LVCMOS33 [get_ports {a}] #normally g


set_property PACKAGE_PIN U2 [get_ports {an0}]
   set_property IOSTANDARD LVCMOS33 [get_ports {an0}]
set_property PACKAGE_PIN U4 [get_ports {an1}]
   set_property IOSTANDARD LVCMOS33 [get_ports {an1}]
set_property PACKAGE_PIN V4 [get_ports {an2}]
   set_property IOSTANDARD LVCMOS33 [get_ports {an2}]
set_property PACKAGE_PIN W4 [get_ports {an3}]
   set_property IOSTANDARD LVCMOS33 [get_ports {an3}]


##Buttons

set_property PACKAGE_PIN T18 [get_ports Cbtn]
   set_property IOSTANDARD LVCMOS33 [get_ports Cbtn]
set_property PACKAGE_PIN W19 [get_ports T1btn]
   set_property IOSTANDARD LVCMOS33 [get_ports T1btn]
set_property PACKAGE_PIN T17 [get_ports T0btn]
   set_property IOSTANDARD LVCMOS33 [get_ports T0btn]
set_property PACKAGE_PIN U17 [get_ports rst]
        set_property IOSTANDARD LVCMOS33 [get_ports rst]

```